MeTube ABR Experiment Tools

Summary guide · browser-side CSV recording and offline graph generation

Purpose

These tools make it possible to observe HLS adaptive-bitrate (ABR) behavior while a long MeTube video plays. They are intended for controlled project experiments: record the player's fragment downloads, export a CSV, then render a consistent figure offline without replaying the video.

1. Browser recorder — isolated route

The new **/experiment/<video-id>** route has its own video element and hls.js instance. Normal viewing at **/watch/<video-id>** is not instrumented, and its player configuration is not changed. The normal watch page only gains a link to open the lab.

When recording starts, the lab resets playback to the beginning. It listens for HLS fragment-loaded and fragment-buffered events and records one CSV row per completed main-video segment. Each row includes:

Field group	What is captured
Timeline	Experiment elapsed time, segment media start/end, download start/end, segment duration.
Rendition	HLS level index, resolution height, advertised variant bitrate.
Transfer	Bytes loaded and measured fragment throughput (bytes × 8 / loading time).
Playback	Buffered state, whether the playback head was observed in the segment, current buffer end.
Estimates	hls.js aggregate bandwidth estimate when exposed by the installed version, plus two companion EWMAs.
Context	Video ID/title and a user-editable network-profile label.

At the end of playback, a CSV download is attempted automatically (if enabled). The user can also stop early or download current data manually. Browser settings may affect whether the automatic download is accepted.

Important estimator distinction

The fast and slow curves use a documented companion EWMA calculated from fragment-throughput samples with 3 s and 9 s half-lives. They are **not represented as hls.js's private internal EWMA values**. The separate hls.js estimate column is recorded where the installed version exposes it. Fragment throughput is an application-layer sample, not a continuous measurement of physical network capacity.

2. Offline graph generator

The Python script **tools/abr/plot_abr.py** reads the exported CSV and writes a high-resolution PNG and a PDF with the same basename. It uses Matplotlib; it does not need to contact MeTube, the browser, or the network. Replotting with different labels or styling therefore does not require replaying a long video.

The x-axis is the video's media timeline. Each segment's rectangle spans its actual playlist duration. Rendition bands are derived from the advertised bitrates recorded in the CSV. Red cells mark segments observed playing; blue cells mark segments reported buffered but not observed playing. The measured throughput, fast EWMA, slow EWMA, and hls.js estimate are drawn over the same bitrate axis.

Quick start

From the repository root, install the Python dependency and pass the downloaded CSV to the plotter:

```
python -m pip install -r tools/abr/plot-requirements.txt
python tools/abr/plot_abr.py /path/to/experiment.csv
```

3. Optional network profile helper

The Linux helper uses **tc** and an IFB interface to shape incoming traffic in phases. Its profile CSV specifies duration, rate in Mbit/s, and a label; use **unlimited** for an uncapped phase. The example profile is illustrative and should be adjusted to the actual rendition bitrates.

Safety warning: traffic-control changes can interrupt connectivity. The helper intentionally requires an explicit acknowledgement, refuses interfaces where it detects an existing nontrivial qdisc, and removes its temporary rules on exit. It does not reconstruct arbitrary previous qdisc configuration. Use only on a dedicated test interface or disposable test machine, after reading **tools/abr/README.md**.

Suggested workflow

1. Pick a ready 60–90 minute video.
2. Open the experiment route and start recording.
3. Run a reproducible network profile schedule on the machine receiving the stream; change the profile label in the lab when the phase changes.
4. Keep the downloaded CSV and the shaping log together.
5. Run the offline plotter and use the PNG/PDF in the report.

Interpretation limits

The recorder depends on hls.js events and is not designed for native-HLS-only playback such as Safari's native path. The plotted throughput can vary due to HTTP overhead, server load, caching, and competing traffic. For rigorous comparisons, document the browser and hls.js version, interface, video ID, profile schedule, date, and repeat each experiment.

Files in the integration ZIP preserve repository-relative paths. See INTEGRATION.md for installation steps and tools/abr/README.md for the full operational notes.